

Speech-Recognition

Midterm 01 Speech Recognition Lab, Part of LN / midterm, see iLearn course!

Prof. Dr.-Ing. Udo Garmann

1 Introduction

In this practical lab we will explore the *classification* of one or more spoken words. The goal is to develop a model that correctly assigns a self-recorded speech command to a predefined class. The data source is the “Speech Commands dataset” by Warden (2018), which contains numerous short speech recordings of simple commands (e.g., “yes”, “no”, “up”, “down”). We will use Python as the programming language and employ scikit-learn for model development.

2 Learning Objectives

- Understanding of audio data preprocessing.
- Extraction of relevant features from audio signals (e.g., MFCCs).
- Application of classification algorithms to speech data.
- Evaluation of speech classification models.
- Practical experience working with the Speech Commands Dataset.

3 Theoretical Foundations (Repetition, see slides/spr-02-speech.pdf)

3.1 Audio Data and Its Representation

Digital audio data consists of discrete samples of a continuous sound signal. Important concepts include:

- **Sampling Rate:** Number of samples per second (e.g., 16 kHz). Higher sampling rates allow for the capture of higher frequencies.
- **Bit Depth:** Number of bits per sample, which determines the amplitude resolution (e.g., 16 bits).
- **Waveform:** Graphical representation of the amplitude over time.

3.2 Feature Extraction for Speech Signals

Raw audio signals are often too complex for direct classification. Therefore, we extract features that compress the relevant information.

- **Mel-Frequency Cepstral Coefficients (MFCCs):** These are the most commonly used features in speech recognition. They are based on the assumption that the human ear perceives frequencies logarithmically. MFCCs represent the spectral envelope of the signal and are robust against noise and speaker variation.
- **Typical Steps to MFCC Calculation:**
 1. **Framing:** Dividing the signal into short, overlapping windows.
 2. **Windowing:** Applying a window function (e.g., Hamming window) to reduce discontinuities at the window boundaries.

3. **Fast Fourier Transform (FFT):** Converting each window from the time domain to the frequency domain (spectrum).
4. **Mel Filter Bank:** Applying a filter bank with filters that are logarithmically distributed on the Mel scale.
5. **Logarithmic Scaling:** Applying the logarithm to the energy of the filter bank outputs.
6. **Discrete Cosine Transform (DCT):** Conversion of log-mel spectra to the cepstral range. The first coefficients are the MFCCs.

3.3 Classification Algorithms

We will use one or more of the following different algorithms from **scikit-learn**:

- **Support Vector Machines (SVMs):** Effective for high-dimensional data.
- **K-Nearest Neighbors (KNN):** Simple, non-parametric algorithm.
- **Random Forest:** Ensemble method that combines many decision trees.

4 The Speech Commands Dataset

The complete Speech Commands Dataset contains thousands of 1-second audio files of 30 different words. We use a subset of these words with 1 or more commands. The data is divided into three main categories:

- **Training Data:** For training the model.
- **Validation Data:** For fine-tuning hyperparameters and verifying generalization during training.
- **Test Data:** For final testing of the model on unseen data.

The most frequent words in the dataset are “yes,” “no,” “up,” “down,” “left,” “right,” “on,” “off,” “stop,” and “go.” Background noise and unknown words are also included, which are important for a robust model.

5 Task Description

The lab is divided into several phases:

1. **Preparation:** Setting up the development environment and downloading the dataset.
2. **Data Exploration and Preprocessing:** Loading and analyzing the audio data, calculating MFCCs.
3. **Model Training:** Implementing and training a classification model. Start with a binary classification of one command, e.g. Support Vector Classification (SVC) with the command “up”.
4. **Evaluation:** Evaluating the model’s performance.
5. **Individual Recording and Classification:** Recording an instruction and classifying it using the trained model.

The task of the midterm is to implement at least a binary classification with one classifier on one command (e.g. Support Vector Classification (SVC) with command “up”). There must be at least 5 recordings that must be classified.

You may extend your basic approach to classify a single command against several commands (e.g. SVC with commands “up”, “down”, “left”, “right”). You may also try different classification algorithms like Random Forest Classifier and K-Nearest Neighbors Classifier.

6 Practical Implementation

6.1 Preparation (see spr-ex-01-intro.pdf)

Step 1: Setting up the Development Environment

Make sure you have Python and the following libraries installed:

- numpy
- scipy
- matplotlib
- librosa (for MFCCs)
- soundfile (for loading WAV files)
- scikit - learn

Installation via pip:

```
pip install numpy scipy matplotlib librosa soundfile scikit-learn
```

Step 2: Download the Speech Commands Dataset

The dataset is large (>2 GB). You can download it directly from the TensorFlow Datasets page or manually:

```
import tensorflow_datasets as tfds
```

Alternatively, manually download from:

http://download.tensorflow.org/data/speech_commands_v0.02.tar.gz

Extract it to a directory of your choice, e.g., './speech_commands'

For this tutorial, it is recommended to use the manually downloaded files, as `tensorflow\_datasets` often pre-processes the data, which can be a hindrance for your own steps. Save the extracted dataset in a folder, e.g., `speech\_commands\_v0.02`.

6.2 Data Exploration and Preprocessing (Single/Pair)

Task 1: Loading and Playing an Audio File

Load any audio file from the dataset (e.g., `speech_commands_v0.02/yes/0a7c2a8d_nohash_0.wav`) and play it.

Note: `librosa .load()` loads the file and returns the sample rate.

```
import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np
import soundfile as sf # Alternative to librosa.load for direct access to audio data

# Path to a sample file (adjust)
audio_file_path = 'speech_commands_v0.02/yes/0a7c2a8d_nohash_0.wav'

# Loading the audio file
# audio_data, sample_rate = librosa.load(audio_file_path, sr=None) # sr=None to keep the
# original sample rate
audio_data, sample_rate = sf.read(audio_file_path) # Alternative for pure data access

print(f"Sample rate: {sample_rate} Hz")
print(f"Number of samples: {len(audio_data)}")
```

```
print(f"Duration: {len(audio_data) / sample_rate: .2f} seconds")

# Visualize waveform
plt.figure(figsize=(12, 4))
librosa.display.waveshow(y=audio_data, sr=sample_rate)
plt.title('Waveform of the audio file')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.show()

# Play audio file (if in Jupyter Notebook)
Audio(data=audio_data, rate=sample_rate)
```

Task 2: MFCC Extraction

Use the following function (file code_extract_mfccs.py) that reads an audio file and calculates its MFCCs. Experiment with different parameters such as `n_mfcc` (number of coefficients), `n_fft` (window size), and `hop_length` (step size).

```
def extract_mfccs(audio_path, sr=16000, n_mfcc=256, n_fft=4096, hop_length=512):
    """ Extracts MFCCs from an audio file.
    Args:
        audio_path (str): Path to the audio file.
        sr (int): Target sampling rate (default: 16000 Hz for speech commands).
        n_mfcc (int): Number of MFCCs to extract.
        n_fft (int): Size of the FFT window.
        hop_length (int): Step size between consecutive frames.

    Returns:
        numpy.ndarray: Array of MFCCs (shape: n_mfcc, number_of_frames).

    saved file names convention:
    Figure1_yes_sr16000_n_mfcc13_n_fft2048_hop_length512.png
    """

    audio, _ = librosa.load(audio_path, sr=sr)

    mfccs = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=n_mfcc, n_fft=n_fft,
                                hop_length=hop_length)
    return mfccs

# Example of MFCC extraction and visualization
mfccs = extract_mfccs(audio_file_path)
print(f"Shape of the MFCCs: {mfccs.shape}")

plt.figure(figsize=(10, 4))
librosa.display.specshow(mfccs, x_axis='time', sr=sample_rate, hop_length=512)
plt.colorbar()
plt.title('MFCCs')
plt.tight_layout()
plt.show()
```

Task 3: Preparing the Dataset for Training

Start with the binary classification of a single command.

As an extension you may create a classifier for several (2-10) commands.

Use the following code (file `code_train.py` to train your models.

Suggestion for scikit-learn: Average the MFCCs over time to obtain a feature vector of dimension `n_mfcc`.

```
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Main directory of the dataset (adjust)

data_dir = 'speech_commands_v0.02'

def prepare_dataset_scikit_learn(data_directory, sr=16000, n_mfcc=13):
    X = [] # Features (MFCCs)
    y = [] # Labels
    """
    In this example ten commands are use for the training.
    As a first step we recommend to start with a binary classification. For this remove
    the
    loop or reduce the list of allowed words.
    """
    allowed_words = ['yes', 'no', 'up', 'down', 'left', 'right', 'on', 'off', 'stop',
                     'go']
    # You can also add _background_noise_ and _unknown_
    for root, _, files in os.walk(data_directory):
        for file in files:
            if file.endswith('.wav'):
                label = os.path.basename(root)
                if label in allowed_words:
                    file_path = os.path.join(root, file)
                    try:
                        mfccs = extract_mfccs(file_path, sr=sr, n_mfcc=n_mfcc)
                        # For scikit-learn: Average MFCCs over time
                        mfccs_mean = np.mean(mfccs, axis=1)
                        X.append(mfccs_mean)
                        y.append(label)
                    except Exception as e:
                        print(f"Error processing {file_path}: {e}")

    # Convert to numpy arrays after collecting all files
    X = np.array(X)
    y = np.array(y)

    if X.size == 0:
        raise ValueError(f"No audio samples found in {data_directory}")

    # Convert labels to numeric values
    label_encoder = LabelEncoder()
    y_encoded = label_encoder.fit_transform(y)

    # If we only have a single sample, return it as the training set and empty others
    if len(X) == 1:
        X_train = X
        y_train = y_encoded
        n_features = X.shape[1] if X.ndim == 2 else 1
        X_val = np.empty((0, n_features))
        X_test = np.empty((0, n_features))
```

```

y_val = np.empty((0,), dtype=y_encoded.dtype)
y_test = np.empty((0,), dtype=y_encoded.dtype)
return X_train, X_val, X_test, y_train, y_val, y_test, label_encoder

# First split: train vs (val+test)
try:
    X_train, X_temp, y_train, y_temp = train_test_split(
        X, y_encoded, test_size=0.3, random_state=42, stratify=y_encoded
    )
except ValueError:
    # Fallback to non-stratified split when stratification is not possible
    X_train, X_temp, y_train, y_temp = train_test_split(
        X, y_encoded, test_size=0.3, random_state=42, stratify=None
    )

# If there is 0 or 1 sample left in X_temp, assign appropriately
if len(X_temp) == 0:
    print("Length of X_temp == 0. Assign appropriately")
    n_features = X.shape[1] if X.ndim == 2 else 1
    X_val = np.empty((0, n_features))
    X_test = np.empty((0, n_features))
    y_val = np.empty((0,), dtype=y_encoded.dtype)
    y_test = np.empty((0,), dtype=y_encoded.dtype)
    return X_train, X_val, X_test, y_train, y_val, y_test, label_encoder

if len(X_temp) == 1:
    # Put the single remaining sample into validation and leave test empty
    print("Length of X_temp == 1.")
    X_val = X_temp
    y_val = y_temp
    n_features = X.shape[1] if X.ndim == 2 else 1
    X_test = np.empty((0, n_features))
    y_test = np.empty((0,), dtype=y_encoded.dtype)
    return X_train, X_val, X_test, y_train, y_val, y_test, label_encoder

# Split the temporary set into validation and test
try:
    X_val, X_test, y_val, y_test = train_test_split(
        X_temp, y_temp, test_size=0.5, random_state=42, stratify=y_temp
    )
except ValueError:
    X_val, X_test, y_val, y_test = train_test_split(
        X_temp, y_temp, test_size=0.5, random_state=42, stratify=None
    )

return X_train, X_val, X_test, y_train, y_val, y_test, label_encoder

print("Prepare data for scikit-learn...")
X_train_sk, X_val_sk, X_test_sk, y_train_sk, y_val_sk, y_test_sk, label_encoder_sk =
    prepare_dataset_scikit_learn(data_dir)

print(f"Training data Shape (scikit-learn): {X_train_sk.shape}, Labels: {y_train_sk.shape}")
print(f"Validation data Shape (scikit-learn): {X_val_sk.shape}, Labels: {y_val_sk.shape}")
print(f"Test Data Shape (scikit-learn): {X_test_sk.shape}, Labels: {y_test_sk.shape}")
print(f"Classes: {label_encoder_sk.classes_}")

```

6.3 Model Training

Task 4: Model Selection and Training

Choose a classification model from scikit-learn.

Start with the training of a single classifier. Take one of the following: Support Vector Machine (SVM), Random Forest, or K-Nearest Neighbors (KNN) classifier.

Train a Support Vector Machine (SVM), Random Forest, or K-Nearest Neighbors (KNN) classifier.

Save the model into a joblib file, so that it can be used later.

```
from sklearn.svm import SVC

from sklearn.ensemble import RandomForestClassifier

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import classification_report, accuracy_score

# Example: Training a Support Vector Machine (SVC)

print("\nTraining an SVC model (scikit-learn)...")

svm_model = SVC(kernel='rbf', C=10) # Experiment with Kernel and C

svm_model.fit(X_train_sk, y_train_sk)

# Evaluation

y_pred_svm = svm_model.predict(X_test_sk)

print("SVC Classification Report:")

print(classification_report(y_test_sk, y_pred_svm,
    target_names=label_encoder_sk.classes_))

print(f"SVC Accuracy on Test Data: {accuracy_score(y_test_sk, y_pred_svm):.4f}")

# Example: Training a Random Forest Classifier

print("\nTraining a Random Forest Model (scikit-learn)...")

rf_model = RandomForestClassifier(n_estimators=100, random_state=42) # Experiment with
    n_estimators

rf_model.fit(X_train_sk, y_train_sk)

# Evaluation

y_pred_rf = rf_model.predict(X_test_sk)

print("Random Forest Classification Report:")

print(classification_report(y_test_sk, y_pred_rf, target_names=label_encoder_sk.classes_))

print(f"Random Forest Accuracy on Test data: {accuracy_score(y_test_sk, y_pred_rf):.4f}")
```

6.4 Own recording and classification

Task 5: Recording Your Own Command

Record a short voice command (approximately 1 second) that corresponds to the words in the dataset (e.g., “yes” or “no”). Save the recording as a WAV file with a sampling rate of 16 kHz.

Start with 5 recordings of 1 command. If you have a trained model for several commands, record these also.

Note: You can use (online) recording tools (like Audacity) or Python libraries such as `sounddevice` or `pyaudio`.

Example for recording with `sounddevice` (if installed):

```
import sounddevice as sd
import soundfile as sf

# Recording parameters
fs = 16000 # Sample rate
duration = 1 # Seconds
channels = 1 # Mono

print("Please say one word now (e.g., 'yes', 'no', 'up'). Recording will start in 3
      seconds...")
sd.sleep(3000) # 3-second wait

my_recording = sd.rec(int(duration * fs), samplerate=fs, channels=channels,
                      dtype='float64')
sd.wait() # Wait until recording is complete

# Save the recording
output_filename = 'my_command.wav'
sf.write(output_filename, my_recording, fs)

print(f"Recording saved as {output_filename}")
```

Task 6: Classifying Your Own Recording

Load your own recording, extract the MFCCs in the same way as for the training set, and classify them using your trained model.

```
# Path to your own recording
my_audio_path = 'my_command.wav'

# Extract MFCCs

# Important: Use the same parameters as during training!
my_mfccs = extract_mfccs(my_audio_path, sr=16000, n_mfcc=13)

# Select model (scikit-learn or TensorFlow) and make prediction

# --- If scikit-learn model was used ---

# Average MFCCs and reshape for scikit-learn
my_mfccs_processed_sk = np.mean(my_mfccs, axis=1).reshape(1, -1)

predicted_label_index_sk = svm_model.predict(my_mfccs_processed_sk)[0] # Or
                             rf_model.predict

predicted_word_sk = label_encoder_sk.inverse_transform([predicted_label_index_sk])[0]
```

```
print(f"\nScikit-learn model says: {predicted_word_sk}")
```

7 Submission and Assessment

Submit the Python scripts (.py) that contain all the steps of the lab. Also submit your own voice recordings. Add a PDF document with your descriptions must be included. Zip these into a single ZIP-archive and upload it in iLearn.

Assessment Criteria:

- Correct implementation of the functions / MFCC extraction.
- Successful training of a classification model.
- Correct evaluation of the model.
- Successful classification of your own recordings.
- A written summary in PDF format of your results and observations. At least answer the following questions:
 - Was a data preparation necessary? Which one?
 - What classification model was best for your task?
 - What accuracy was achieved?
 - What difficulties were encountered during your recording?

8 Extension Options (Optional)

- Experiment with different classification algorithms and hyperparameters.
- Try other features (e.g., spectrogram, chroma features).
- Add noise to the training data to improve the model's robustness.
- Implement a pipeline for the entire processing workflow (from acquisition to prediction).
- Visualize the confusion matrix for the test data.

Good luck with the lab!